

W2-Ex3 — Collections, ingestion, and the retrieval gate

Demos `04-collections-explorer`, `simple-dynamic-rag` and `05-embeddings-rag-explorer`, all on local Ollama — `omic-embed-text` for the vectors, `qwen3:1.7b` for the answers.

1 — A COLLECTION IS A DATABASE

What persisted between the two runs was every chunk together with its embedding and metadata: the second run listed not only the four chunks about electricity tariffs I had just added, but also four chunks about football I had stored in a session hours earlier — it lives in `./collections-store`, a ChromaDB database on disk (a `chroma.sqlite3` file for chunks and metadata plus a folder holding the vector index). Querying with words the chunks never use worked as promised: *"quin equip va quedar campió de la competició europea?"* surfaced *"El psg va guanyar la champions la temporada passada"* first at similarity **0.651**, sharing no keyword. Re-inserting an existing chunk is refused: ids are `source::chunk_number`, so the collection blocks silent duplicates.

2 — ONE DOCUMENT, TWO CHUNKINGS

I ingested a real work document — `architecture.md`, the architecture guide of a microservice I work on (~26k characters, Spanish, 16 `##` sections). The `chars` strategy produced **38 chunks** (800-char window, 100 overlap); `sections` produced **17 chunks**. Same three questions to both:

QUESTION	BYCHARS — TOP HIT	BYSECTIONS — TOP HIT
¿cómo se manejan los errores?	0.623 – table fragment about Keycloak credentials	0.675 – section “7. Manejo de errores”
¿cómo se gestionan las sesiones de BD?	0.578 – window ending mid-code-block	0.608 – section “8. Gestión de sesiones”
¿qué medidas de seguridad hay?	0.578 – user-sync table fragment	0.615 – section “1. Estilo arquitectónico”

Sections retrieved better on all three questions — it lands on the exact section, with higher similarity, because a section chunk carries its own heading and one complete idea, while an 800-char window cuts mid-table and mid-code-block. The difference showed hardest in the answers: on the security question, `bychars` handed the model four table fragments and the model padded the gaps by *inventing* AES-128 encryption, AWS Shield and FIFO queues — none of which the document mentions — while `bysections` stayed on what the document actually says (JWT validated against Keycloak's JWKS, `NodeValidator` permissions, Vault for secrets). The price of sections is size: 2.1–2.5k prompt tokens per turn versus 1.1–1.7k for chars.

3 — THE THRESHOLD BAND

Against the `bysections` collection, real questions peaked at **0.675** and **0.608**, while off-topic ones did *not* score near zero: a paella recipe reached **0.505** and a question about back-pain exercises reached **0.552** — the second one quietly breaking a threshold of 0.55 that the first had made look safe.

real	errores	0.675	0.595	0.585	0.578	sesiones BD	0.608	0.566	0.563	0.561
off-topic	paella	0.505	0.490	0.477	0.470	dolor espalda	0.552	0.546	0.545	0.545
band found: threshold ≈ 0.57 (blocks both off-topic: every row X below threshold; both real questions still retrieve) · at >0.608 the sessions question starves too										

Why this number cannot be hard-coded: the band belongs to this embeddings model, this document and this language — my whole corpus lives in a 0.50–0.68 strip, so 0.57 separates on-topic from off-topic here, but a different model, corpus or phrasing shifts the strip, and my own second off-topic question already moved the ceiling from 0.505 to 0.552. A threshold is a calibration against *your* data, not a constant — and it will not catch everything, which is why the grounded system prompt stays as the second line of defense.